

Software Techniques for Advanced Hotspot Prediction and Mitigation

An Honors Thesis for the Department of Electrical and Computer Engineering

Jacob Merritt

Tufts University, 2026.

Contents

Abstract	1
Introduction	2
Background	5
A. Current State of Code-Level Research	5
B. HotGauge	6
C. How Software-Microarchitecture Interaction Creates Thermal Behavior	7
Chapter 1: Can hotspots be eliminated using only software techniques?	9
Sensitivity Study	9
Experimental Setup	9
Results	12
Chapter 2: Can we automate the process of identifying, quantifying, and extracting computational behaviors that affect hotspot performance?	16
Feature Extraction	16
SniperSim Features	17
QEMU Features	18
QEMU and SniperSim Derived Features	18
Experimental Setup	19
Results	19

Chapter 3: Can we predict the medium-term hotspot performance of a program from software-generated features?	25
Motivation	25
Prediction Model Limitations	25
Experimental Setup	26
Results	26
Opportunities for further research	28
Conclusion	29
Bibliography	30

Abstract

Modern microprocessors are increasingly susceptible to advanced hotspots which form faster than thermal sensors can detect and whose localized nature renders previous methods of chip cooling, particularly cooling the entire chip with air or water, inefficient. These advanced hotspots affect device performance and reliability and, with current cooling solutions, balloon energy costs. Previous hotspot-mitigation techniques have primarily treated advanced hotspots as an external thermal management problem and have not addressed the medium-to-long term computational behaviors that give rise to them. This work focuses on what we can know about the hotspot performance of a program without running a detailed thermal simulation on it, paving the way for future hotspot-aware compilers and operating systems.

In this paper, we show that software techniques alone can impact advanced hotspot formation and severity. To identify the specific computational behaviors that lead to advanced hotspot formation, we present an automated feature extraction pipeline that runs a program through two CPU emulators and extracts information about its instruction makeup, dependency structure, and memory behavior. Assigned to these feature vectors are labels consisting of the recently introduced hotspot metrics hotspot-severity and Maximum Local Temperature Difference (MLTD). A random forest model trained on these feature vectors achieved approximately 80% classification accuracy on the available interval dataset. However, because the dataset contains few high-severity hotspot examples, this result should be interpreted as evidence of predictive promise rather than as a complete hotspot-formation predictor.

Introduction

Until the mid-2000s, CMOS technology largely followed Dennard scaling, under which reductions in transistor dimensions were accompanied by proportional reductions in supply voltage. This meant that engineers could squeeze more, smaller transistors onto a chip without feeding the chip more power. As a result, transistor counts exploded and clock frequencies grew. During the mid-2000s, transistors reached a size at which the proportional supply voltage reduction made CMOS leakage power a first-order term. This meant that with each new technology node, power density increased. It also meant that raising clock frequency required raising power proportionally.

Growing power consumption increased both the steady-state temperature of chips and the speed at which temperature accumulated during operation. At higher temperatures, transistors switch more slowly, limiting the speed at which the clock can run. Additionally, higher temperatures increase leakage current. In a phenomenon called “thermal runaway”, this leakage current draws more power which, forming a positive feedback loop, increases the temperature of the chip [1]. In the past, this problem was solved by blowing air over the chip’s heatsink. Modern chips have required more expensive and energy-intensive liquid cooling. Additionally, engineers have had to pair liquid cooling with system-wide techniques like power gating and dynamic voltage and frequency scaling (DVFS) [2]. Together, these techniques can alternate which parts of the chip are running and which are resting and cool the entire chip.

While valuable, this solution is ultimately inadequate to address the thermal regulation of computer chips in the long-term because temperature increases on chips are not spread in a uniform manner. They instead occur locally and intensely

in what are called advanced hotspots. As these hotspots increase in peak temperature, it has become prohibitively inefficient to cool them using only techniques that cool the entire chip. Furthermore, as the speed at which hotspots form quickens, using temperature sensors to determine when to dynamically change the workloads of different blocks of logic has become too slow to prevent hotspots before they develop [2].

To address the challenges posed by localized and rapidly forming thermal hotspots, a range of hotspot-focused mitigation techniques have been proposed. One approach seeks to improve the efficiency of cooling by doping wafers in ytterbium ions and “zapping” hotspots away with lasers using anti-Stokes cooling [3]. Another predicts hotspot formation in advance using machine learning models trained on workload characteristics and applies DVFS [2]. While effective in cooling chips, these techniques rely on building new hardware or cooling systems, presenting an opportunity for more lightweight advanced hotspot mitigation solutions that instead focus on the underlying computational behaviors that cause the hotspots to appear.

Hotspots emerge from specific instruction patterns and sequences executing on various parts of a computer chip. The choices of software developers and compilers thus affect hotspot performance. In a controlled experiment, we found that reducing the instructions per cycle on a synthetic benchmark from 1.15 to .75 could eliminate a debilitating hotspot on a chip.

To identify how the construction of programs leads to different developments of advanced hotspots, we run benchmarks through two CPU emulators and extract performance statistics. We first collect detailed instruction and memory behavior from SniperSim, a cycle-level, interval-based microarchitectural simulator. We then run the program through QEMU, a higher-level emulator, to break down executables into a series of basic block traces. The assembly code of each trace

is recreated from the disassembly of the original executable, allowing us to extract entropy and intra-trace dependency features.

We collect these features on small code intervals of SPEC benchmarks and then feed a looped version of these intervals through the HotGauge simulation pipeline to obtain the medium-term thermal behavioral statistics of only those intervals of code. We perform an ablation study and SHAP feature analysis on these features and find that the SniperSim microarchitectural counters are more predictive of medium-term thermal behavior than the QEMU-generated structure features. We also find that features describing throughput and the intensity of program execution are particularly predictive of thermal behavior.

A random forest machine learning model is trained on the dataset and can predict three classes of hotspot behavior with 80% accuracy. While flawed in important ways, this model demonstrates promise for future hotspot predictors.

Background

A. Current State of Code-Level Research

There have been some efforts to utilize operating system (OS) and compiler techniques to reduce hotspots. The prominent OS strategies include assigning tasks to cores in a balanced manner, thermal-aware scheduling, and running “cool loops” to allow the chip to briefly rest. Prominent compiler strategies include inserting nop instructions and adjusting instruction binding based on which parts of the device are hot [4].

The issue with these technique is how the researchers built their OS or compiler such that it can determine which sections of code are hot enough to warrant the insertion of cool loops and nop instructions and which tasks are “hot jobs” that should be specially scheduled or sent to a different core. There are two broad ways this has been done. The OS scheduling techniques in [7][8] rely on thermal sensors on the chip. In [7], tasks are run through a thermal simulation which gives the task a value corresponding to how much time a core running that task would be above a certain threshold.

These methods for identifying “hot jobs” are outdated in important ways. On modern CPUs and GPUs, hotspots form more quickly than temperature sensors can detect them [3]. Furthermore, core-level thermal simulations obscure the localized nature of hotspot formations. There is therefore a need for research that moves beyond temperature sensors and core-based simulations to examine how developer and compiler decisions like instruction selection, data structure choice, and code organization influence the formation, location, and severity of thermal hotspots. There is an additional need for hotspot predictors that allow engineers to reliably

identify hot and cool jobs without running detailed thermal simulations that can take days to execute.

One important work, Boreas [2], begins to move in this direction. As a part of their effort to dynamically change the voltage and frequency of a microprocessor in real time, researchers created a gradient-boosted model to predict hotspots in real time. Boreas includes microarchitectural counters as a part of the dataset the model processes. This study is an expansion of their work to focus on the medium-term, rather than short-term, development of hotspots, and to do prediction without live updates from temperature sensors on the chip.

B. HotGauge

HotGauge is the simulation environment for hotspots in modern processors used in this study. It takes in a system model and a workload as inputs, which travel through a performance model, power model, and thermal model to produce time-stamped heat maps [8]. The performance model is SniperSim[10], the power model is McPAT[11], and the thermal model is 3D-ICE[9].

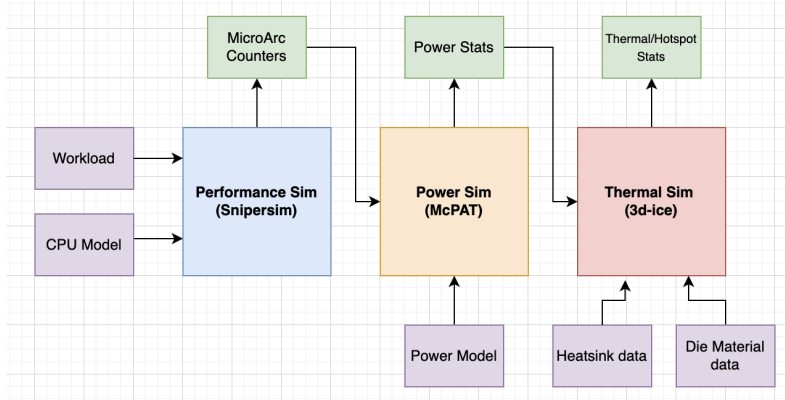


Figure 1: HotGauge Overview

There are several advantages to using HotGauge to test the hotspot performance of different benchmarks. First, HotGauge ties time-stamped thermal information back to time-stamped instruction counts, allowing researchers to know

exactly which instructions were executing at what location during the formation of a hotspot. Furthermore, the output heat maps consist of uniform cells of a user-defined size (typically around 50x50um), allowing for much finer granularity than other simulation tools. Lastly, HotGauge allows researchers to test different technology nodes and different microarchitectures and push them beyond their thermal limits without destroying them.

HotGauge [8] includes novel metrics for the description of hotspots which are used in this work. Specifically, it introduces maximum localized temperature differential (MLTD) which is a measure of gradient intensity, and *hotspot-severity* which combines peak temperature and MLTD into one metric that describes, on a scale from 0 to 1, how dangerous a hotspot is.

C. How Software-Microarchitecture Interaction Creates Thermal Behavior

Hotspot formation is driven by what instructions are executing in each part of the chip and what the current thermal state of that location is. At runtime, instructions are assigned to different microarchitectural units, such as ALUs, FPU, and cache blocks, each of which occupies a distinct physical region of the die. Different instruction mixes, memory-access patterns, and control flows therefore lead to different activation of these units and different hotspot formations.

Compiler and developer decisions like instruction selection, data structure choice, loop transformations, and scheduling directly influence this distribution of instructions across functional units. Even when executing the same function, different implementations of programs can affect different parts of the chip in different orders, leading to unique thermal behavior in each implementation. Prior compiler and OS thermal-aware hotspot-mitigation techniques were unable to examine instruction-level execution because they lacked a capable simulation tool, and instead focused on how groups of instructions affected core-level thermal per-

formance. This work focuses on understanding how software-level decisions propagate through the microarchitecture to influence the formation and severity of thermal hotspots.

Chapter 1: Can hotspots be eliminated using only software techniques?

Sensitivity Study

Before launching into this study, it is important to establish the underlying promise of software-based hotspot mitigation techniques. We do this by constructing a controlled experiment on a synthetic benchmark that isolates real IPC. The experiment creates and simulates benchmarks with a range of real IPCs to evaluate how adjusting this important computational behavior affects hotspot behavior and program performance.

The purpose of this synthetic benchmark is to test if, in a perfect environment, software-only interventions are capable of eliminating hotspots without prohibitive performance cost. If this is true, it justifies extending these techniques to complex, real-world workloads.

Experimental Setup

The most important piece of this controlled experiment is the synthetic benchmark. The benchmark consists of a repeating loop of 500 instructions. At compile time, the user decides what portion of this loop is composed of floating point multiply instructions, and what portion are NOP instructions. The sweep consists of varying the number of instructions in the loop that are NOPs, in this case from 4 to 340.

Algorithm 1 Synthetic NOP and Floating-Point Multiply Benchmark Generation

Require: Total instruction budget per loop iteration $N = 500$

Require: Number of NOP instructions N_{nop}

Ensure: A synthetic benchmark loop with controlled real IPC

- 1: Choose the number of NOP instructions, N_{nop}
- 2: Compute the number of multiply instruction groups:

$$N_{\text{mul_groups}} = \frac{500 - N_{\text{nop}}}{8}$$

- 3: Generate a loop body containing N_{nop} NOP instructions
 - 4: Generate $N_{\text{mul_groups}}$ groups of floating-point multiply instructions
 - 5: **for** each benchmark iteration **do**
 - 6: Execute the NOP instructions
 - 7: Execute the floating-point multiply instruction groups
 - 8: **end for**
-

To compute real IPC, each version of the benchmark is emulated by the SniperSim x86 simulator for 20 million instructions. The model of the CPU used for this sweep is described in table 1. The floorplan is one core to simplify the experiment and isolate thermal behavior to one active region of the die.

Table 1: CPU Microarchitecture Parameters

CPU Microarchitecture Parameters	
Process node [nm]	7
Cores	1
Core area [mm ²]	1.25
Logical CPUs/core	2
Frequency	4.0 GHz
SMT threads/core	2
ROB/window entries	128
LQ entries	72
SQ entries	56
Scheduler entries	64
L1I \$	Private, 32 KiB
L1D \$	Private, 32 KiB
L2 \$	Private, 512 KiB
L3 \$	16 MiB

Producing temperature maps for the workloads requires passing the SniperSim-generated microarchitectural statistic outputs through McPAT to produce time-stamped power files. These files are then fed into 3D-ICE, the temperature model, which simulates temperature based on the power files and the thermal properties of the materials of the die and heatsink.

Table 2: Thermal configuration parameters for the thermal model layers used in this study. The heatsink model used was included with 3D-ICE and models a HS483-ND heatsink with a P14752-ND fan spinning at 6000 rpm.

Layer	Thermal Cond. [W/ $\mu\text{m K}$]	Vol. Heat Cap. [J/ $\mu\text{m}^3 \text{ K}$]	Height [μm]
Thermal grease	0.04e-4	3.376e-12	30
Copper heat spreader	3.9e-4	3.376e-12	3e3
Solder TIM	0.25e-4	1.628e-12	200
Silicon IC wafer	1.20e-4	1.651e-12	380

Results

The sweeps show a consistent decrease in peak temperature and average temperature as the number of NOPs in the benchmark increases and real IPC decreases. Importantly, this relationship is smooth and monotonic across those metrics, with no observed instability.

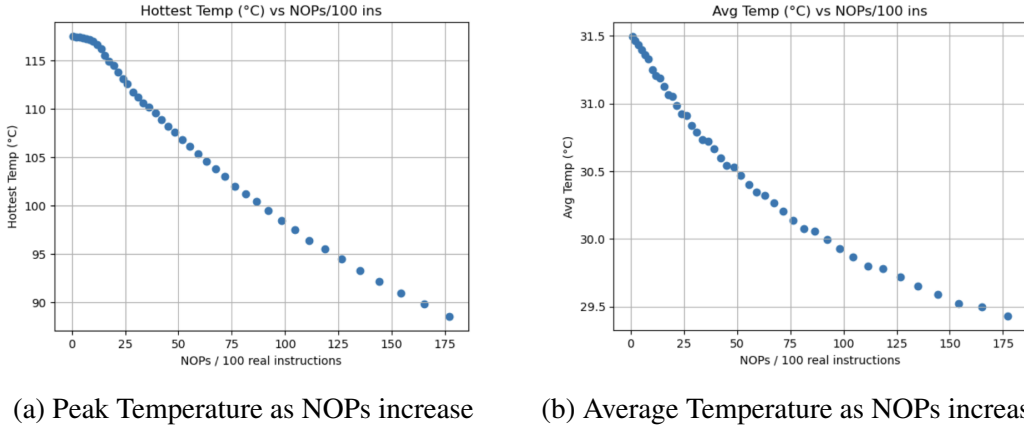


Figure 2: Temperature Behavior Across NOP Sweep

Thermal gradients, as measured by MLTD, also decrease alongside real IPC, but with an interesting threshold effect. Before around 20 NOPs per 100 real instructions, max MLTD values for different runs cluster together. After this threshold, there is a steep drop in MLTD and the monotonic, near-linear relationship

returns.

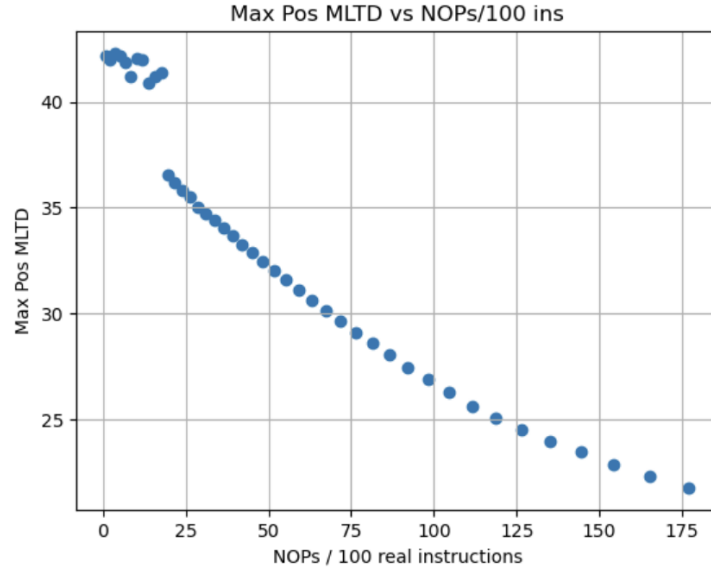


Figure 3: MLTD as NOPs increase

The elimination of the MLTD clustering as NOPs per real instruction increases suggests that NOP insertion in this sweep effectively eliminates a hotspot. To better understand this effect, we examine the time-resolved thermal behavior underlying these samples. The two most important samples for this examination are the two points directly on either side of the threshold. These points are the benchmark that ran at 1.13 instructions per cycle, and the benchmark that ran at 1.12 instructions per cycle. Figure 4 shows the MLTD behavior over time for these two benchmarks.

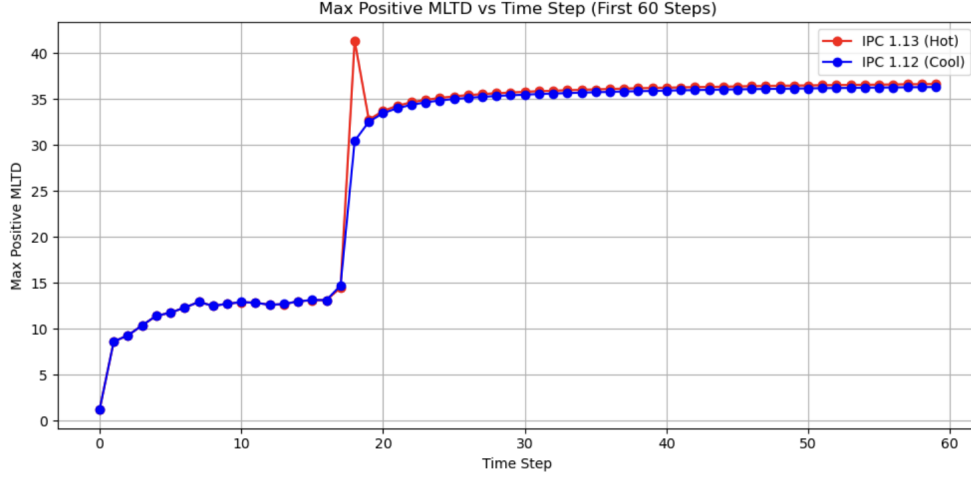


Figure 4: MLTD Over Time Across IPC Threshold

The figure shows that beyond a narrow IPC threshold, there is a large, non-linear spike in the size of the gradient followed by a return to a slower, near-linear build-up of temperature. This large spike is, in effect, a hotspot. It is worth noting that the area on this chart where the two lines closely follow each other is the warmup section and is independent of the benchmark. When MLTD jumps is when the actual instructions start causing thermal behavior.

There is, of course, a performance sacrifice for the elimination of the hotspot. Execution time is not a reliable proxy for program performance in this experiment because the number of instructions is held constant across different compilations of the synthetic benchmark and different compilations do different amounts of work. Therefore, we calculated an expected execution time by normalizing execution times to a constant number of real instructions executed. This expected execution time increased as NOPs were injected. In exchange for limiting hotspot severity from 1 to .66, which allows the processor to run correctly, the overall performance decreased by 38.5%.

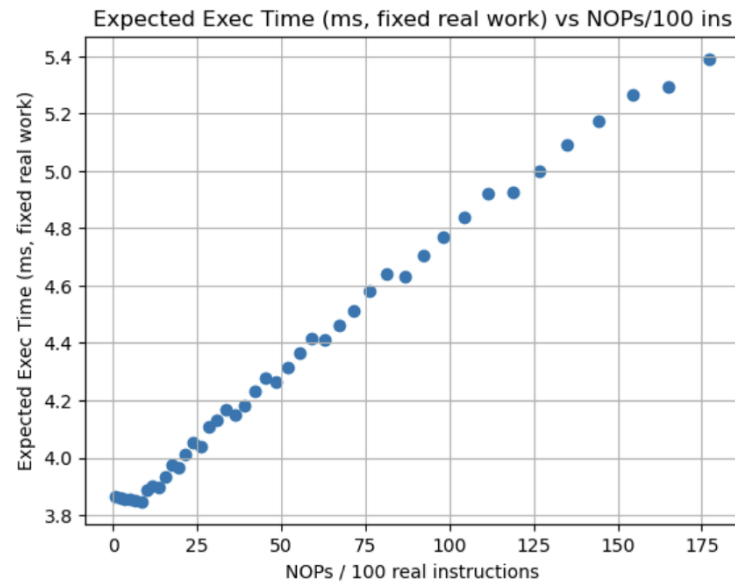


Figure 5: Expected Execution Time Increases as NOPs Are Injected

Chapter 2: Can we automate the process of identifying, quantifying, and extracting computational behaviors that affect hotspot performance?

Feature Extraction

Now that we can assert software can have a dramatic impact on hotspot behavior, it is important to identify the exact software behaviors that impact hotspot formation, establish metrics that quantify their impact, and create methods of extracting these metrics. In order to do this, we need to dissect both the makeup of the program and how it executes on a particular CPU. We propose two pipelines for feature generation, one SniperSim-based and one QEMU-based, and study the relative importance of those features to thermal behavior by using a random forest model to perform ablation and SHAP analysis.

Feature extraction and feature importance analysis are performed on small, 6-20 million instruction intervals of each SPEC benchmark used in the study. The specific intervals are chosen by running a SimPoints analysis on the entire program run. The SimPoints pipeline splits the program into these small intervals and creates a basic block vector (BBV) for each one. A BBV is a histogram consisting of how often each unique basic block in the program executes in a particular interval. A basic block (BB) is a straight-line sequence of instructions with one entry point and one exit point. After a feature reduction on each interval's BBV, SimPoints performs K-means clustering on the intervals to identify which sections of code belong to which phases of program execution. The goal was to select intervals that

belong to different K-means clusters so that the thermal behavior of each interval will be different.

We produce medium-term (after 2 billion instructions) hotspot behavior statistics by running each interval through SniperSim and producing the power statistics for that interval. We then loop these power statistics for as long as it took the original program to execute 2 billion instructions. This new power statistics files are fed into the thermal model which produces the hotspot statistics.

SniperSim Features

The features we generated from SniperSim stem from the microarchitectural counters and functional unit duty cycle statistics that SniperSim produces for a particular period of execution time. From these counters, we took or built three categories of features. The first two groups are taken directly from the SniperSim output files. They are the instruction/memory counters per cycle and the functional unit duty cycles.

The third category is a measure of temporal variation in duty cycle and IPC. We calculate this by examining the duty cycles and IPC for each of the sub-intervals that make up the interval from which we are extracting features. These “sub-intervals” consist of the smallest granularity of program execution from the SniperSim run, in this case .05 ms. We then take the largest sub-interval duty cycle or IPC value and divide it by the average value to gauge the “spikiness” of the interval’s execution and functional unit usage. An exact list of extracted features can be observed in figure x.

These SniperSim microarchitectural counters have been shown to be useful in predicting thermal behavior. Boreas, a previous hotspot prediction model focused on short-term hotspot behavior, was able to reliably predict hotspots based only on thermal sensors and microarchitectural counters [2].

QEMU Features

The features generated from the QEMU emulation are more complex and are intended to recover code structure. When we perform QEMU emulation on a benchmark’s executable, we preserve the execution order, starting program counter (PC), and instruction length of each QEMU translation block (also referred to as a basic block). We then reference the disassembly of the executable to reconstruct the assembly code for each translation block. The end result is a list of assembly instructions the CPU runs in the order in which it runs them.

With this list of assembly instructions, we can construct another three groups of features. The first group of features are instruction mix features. These are QEMU’s view of what instructions executed and how much of the simulated interval they make up. The second group of features are entropy features, specifically basic block entropy and instruction category entropy. Basic block entropy measures how evenly execution is spread across the basic blocks that make up a simulated interval. Instruction category entropy is a measure of how diverse the instruction mix is within a simulated interval. The last group of features are structure features. These include an instruction-level parallelism (ILP) proxy built from the ratio of instructions in a basic block divided by the longest critical path length in the same basic block, summed across all executed basic blocks.

QEMU and SniperSim Derived Features

The seventh and last group of features are the features derived from combinations of SniperSim-generated and QEMU-generated statistics. This feature group consists of three “mismatch” statistics that divide SniperSim-derived duty cycles for the ALU, FPU, and LSU by the QEMU-reported instruction count of instructions that execute in those units.

These statistics measure how intense functional unit activity is relative to

the amount of work that the functional unit has to do. This comparison captures the gap between the instruction stream presented to the processor and the way that stream is realized by the simulated microarchitecture, including effects from stalls, overlap, cache behavior, and pipeline scheduling.

Experimental Setup

To examine the relative importance of these features and feature groups, we perform an ablation study and a SHAP analysis [12] using a random forest machine learning model. We choose a random forest model because they are simple, fast, and handle correlated features better than alternatives. Both the ablation study and the SHAP analysis were performed on the dataset for each of the three output labels, peak temperature, MLTD, and hotspot-severity.

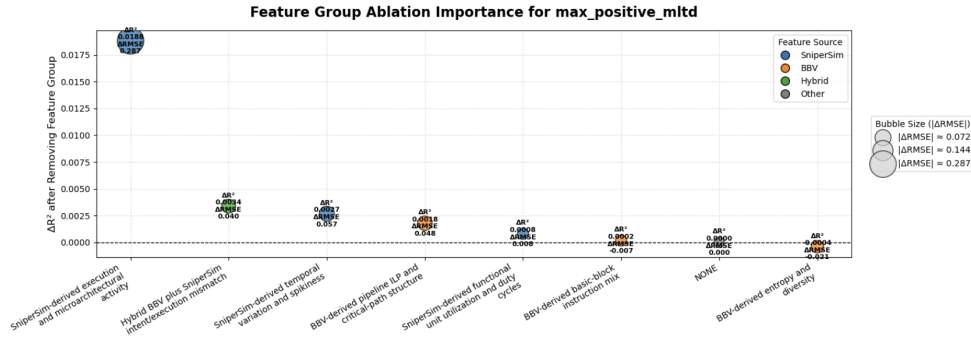
The goal of the ablation study is to compare the relative importance of the seven feature groups. As such, we first train and test the model with all of the features. We then train and test the model with all but one feature group, repeat for all feature groups, and measure the amount that holding out each feature group hurts the model’s predictive power.

The SHAP analysis, on the other hand, is designed to identify individual features and combinations of features that are most predictive of model outcome. A SHAP analysis tracks how much each individual feature pushes an output label up or down. SHAP analyses can do the same thing for pairs of features, which can uncover features that alone are not useful but when coupled with other features become predictive.

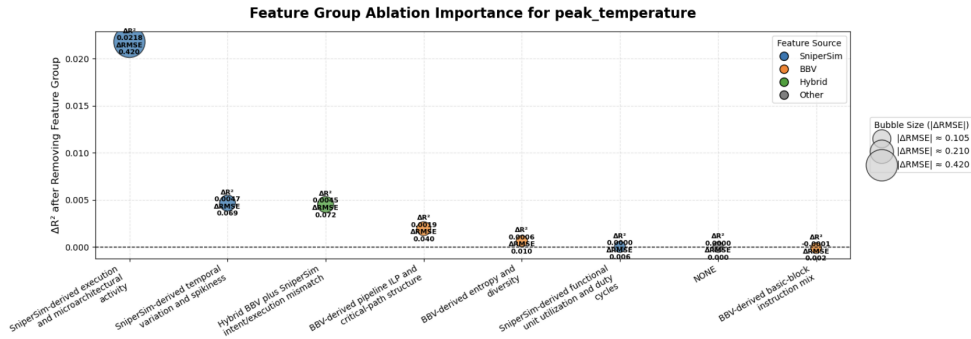
Results

We present the results of the ablation study in a bubble chart. On the x-axis are the feature groups and on the y-axis is ΔR^2 , which measures the loss in explained

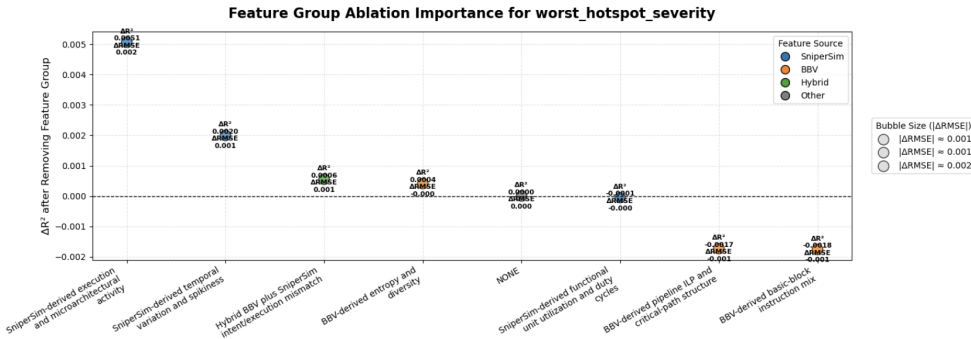
target variance. The diameter of the bubbles is determined by ΔRMSE , which is the increase in prediction error when a feature group is removed



(a) Feature Group Importance to MLTD



(b) Feature Group Importance to Peak Temperature



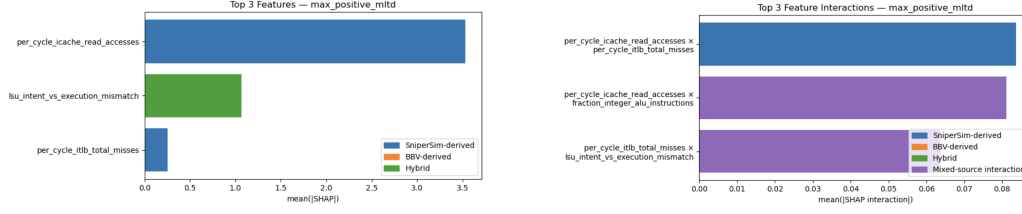
(c) Feature Group Importance to hotspot-severity

Figure 6: Feature Importance for all 3 Hotspot Metrics

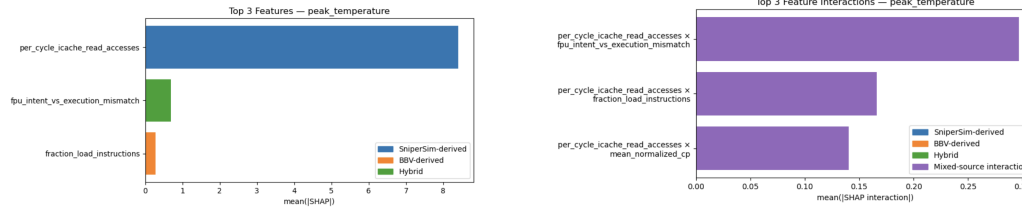
The biggest takeaway from this ablation study is that the microarchitectural counters are by far the most important feature group and the BBV-derived features alone offer almost no predictive capability. It is worth questioning, though, the extent to which this result is a product of the simulation tool used for this study.

HotGauge relies only on SniperSim counters to generate the power statistics that drive the thermal simulations. This means that the HotGauge pipeline does not always simulate the structural aspects of the code that the QEMU-related features are designed to extract. This explains both the relative importance of the SniperSim counters and the nearly complete lack of explanatory power from the QEMU-derived features.

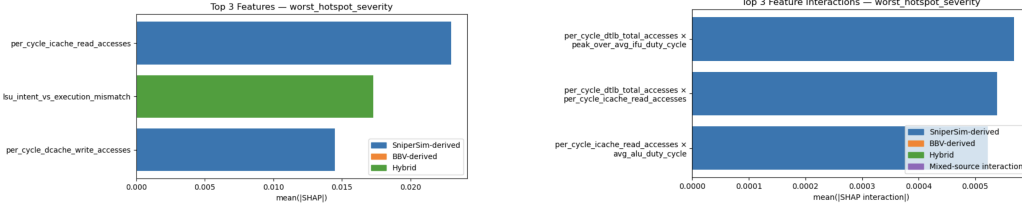
The SHAP analysis results suggest a similar conclusion, but reveal the importance of some of the “spikiness” features that the ablation study misses. Figures x and y display the SHAP analysis outputs. The output metric for the individual feature study is $\text{mean}(|\text{SHAP}|)$, which averages across all samples how much a feature changes the baseline prediction. The output metric for the feature interaction study, $\text{mean}(|\text{SHAP interaction}|)$, does the same thing but with feature pairs.



Feature Importance and Feature Interaction Analysis for MLTD



Feature Importance and Feature Interaction Analysis for Peak Temperature



Feature Importance and Feature Interaction Analysis for hotspot-severity

Figure 7: SHAP Analysis for Feature and Feature Interaction Importance for all Three Hotspot Metrics

There are two interesting takeaways from this SHAP analysis. First, it uncovers the utility of a subsection of QEMU-derived features, particularly when they are combined with SniperSim features. Second, it demonstrates the importance of instruction cache accesses for medium-term thermal performance. This is as opposed to Boreas, which found that over a shorter period of program execution (200 million instructions), ALU accesses and data cache accesses were more predictive of thermal behavior. This comparison, however, should be interpreted cautiously because the prediction horizon, dataset construction, and possibly the thermal setup differ between the two studies.

The QEMU-derived features that turned out to have some predictive power

were the hybrid features that compared duty cycle with the instruction mix metrics and the instruction mix statistics, for example the fraction of instructions that are ALU or load instructions. This is most likely because both the hybrid statistics and the SHAP feature interaction analyses that pair the instruction mix features with SniperSim counter features explicitly link the amount of work a functional unit had to do with the speed at which the functional unit actually did that work. It is reasonable that this relationship is important to thermal behavior as a small amount of work executing quickly might heat up a part of a chip while a small amount of work executing slowly likely will not.

While it is valuable knowledge that this relationship between available work and pace of work has thermal implications, this feature importance analysis does not by itself justify building out the QEMU feature generation pipeline. It is possible that a similar feature could be constructed out of exclusively SniperSim features that would have more predictive power within this simulation framework.

The other important takeaway is the importance of instruction cache read accesses in predicting the medium-term thermal behavior of the instruction intervals. In all three hotspot metrics, the SHAP study determined that instruction cache read accesses were the most predictive feature. This does not necessarily imply a direct causal relationship between instruction cache usage and hotspot formation. Instead, it is likely that instruction cache read accesses are a proxy for instruction throughput. If there is a high rate of instruction fetches, that implies the CPU is processing instructions quickly, and those computations cause the microprocessor to heat up.

It is possible that the importance of instruction cache read accesses as a proxy for throughput increases as program execution grows longer. For longer execution intervals, instruction cache behavior begins to reflect repeated instruction fetches over many cycles rather than shorter startup effects. This could be one rea-

son why instruction cache read accesses were more important in this study than in Boreas.

Chapter 3: Can we predict the medium-term hotspot performance of a program from software-generated features?

Motivation

The ultimate utility of the feature importance analysis we perform in the previous chapter, beyond the research goal of better understanding how hotspots form, is to create more accurate and comprehensive hotspot predictors. The specific contribution of this paper is a prediction model that predicts medium-term hotspot formation using only features derived from software emulators. To this end, we create a random forest model with 500 estimators and no max depth.

Prediction Model Limitations

There are important limitations that affect the utility and accuracy of this prediction model. The most important of these limitations is the dataset, which does not contain samples that report a hotspot-severity score above .62 or a peak temperature above 89.7 degrees Celsius model cannot be said to be a proper hotspot predictor because there are not enough hotspots in the dataset. The second is that it contrasts the findings of the Boreas paper, whose experimental setup this study attempted to replicate. In Boreas, benchmarks like hammer had peak hotspot-severity scores of .96 while running at 4.0 GHz. In this study, no interval extracted from hammer produced a hotspot-severity value above .41. This suggests that there is either something wrong with the simulation pipeline or the experimental setup between the two experiments is different in an important way.

Another important limitation of the model is the model itself. While the random forest model does contain some advantages for this kind of study, for example it handles correlated features well, it was ultimately chosen because it is relatively simple to build and it is fast to train. A gradient boosted tree or a neural network, given more time, would likely produce more accurate results. Additionally, there are only 77 samples which come from only 6 benchmarks, which is likely not enough volume or diversity for a robust predictor.

Experimental Setup

We train and test the random forest model using the same parameters and evaluation framework as in the ablation study and SHAP analysis. There are 500 trees in the forest with no maximum depth and the model was evaluated using a five-fold cross-validation. The model predicts three classes created by dividing the dataset into three quantile bins of hotspot severity. These classes represent low, medium, and high severity relative to this dataset, not absolute severity ranges over the full 0-to-1 hotspot-severity scale.

Results

The results of the k-fold evaluation are described in table 3. The traditional ML model evaluation metrics, precision, recall, and F1-score are shown for each class and are averaged across classes.

Table 3: Pooled classification report across held-out folds.

Class	Precision	Recall	F1-score	Support
0	0.78	0.81	0.79	26
1	0.70	0.64	0.67	25
2	0.89	0.92	0.91	26
Accuracy	–	–	0.79	77
Macro avg	0.79	0.79	0.79	77
Weighted avg	0.79	0.79	0.79	77

These are fairly weak results. The cumulative averages are increased by the model’s accuracy in predicting class two. Since there are no true high-severity hotspot samples in class two, higher accuracy for this class is not notable. In contrast, the poor ability to predict class 1 is worrisome, especially considering the chief task of the predictor is to differentiate between classes 0 and 1. Likely the model simply needs more data.

Opportunities for further research

The most pressing goal for the continuation of this research is creating a richer, more diverse dataset. This includes adding more SPEC benchmarks and more intervals that cover larger ranges of hotspot formation and behavior. It would also improve the quality and utility of the model to vary the frequency at which each benchmark executes and vary the beginning heatmap of the thermal simulation. This would allow for more realistic hotspot predictions as in real computers and data centers no beginning heatmap can be assumed and devices run at different frequencies.

Assuming a more accurate predictor, opportunities for further research explode. One goal of this study that was not accomplished was to create a hotspot-aware NOP-injector that could inject NOPs into hot regions of code as they execute. Other promising areas of research that could make use of this feature extraction pipeline and predictor include hotspot-aware operating systems and compilers.

Conclusion

This study demonstrated the promise of software-based methods for hotspot mitigation. In a controlled environment we demonstrated how NOP injection can eliminate a hotspot. We then created a feature-extraction pipeline to identify the computational behaviors that have the largest impact on thermal behavior. We found that instruction cache read accesses, likely as a proxy for instruction-level throughput, is the microarchitectural counter most predictive of thermal behavior over an execution period of 2 billion instructions. We additionally found that “burstiness” features that compare the work a functional unit needs to do with the average usage of that functional unit are predictive of hotspot behavior. Lastly, we built a predictor that shows the promise of using software-extracted features to predict when during a program’s execution hotspots will form.

Bibliography

- [1] K. Skadron, M. R. Stan, W. Huang, S. Velusamy, K. Sankaranarayanan, and D. Tarjan, “Temperature-aware microarchitecture,” in *Proc. 30th Annual International Symposium on Computer Architecture (ISCA)*, 2003, pp. 2–13, doi: 10.1145/859618.859620.
- [2] M. Amiraski, D. Werner, A. Hankin, J. Sebot, K. Vaidyanathan, and M. Hempstead, “Boreas: A Cost-Effective Mitigation Method for Advanced Hotspots using Machine Learning and Hardware Telemetry,” in *Proc. 2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2023, pp. 295–305, doi: 10.1109/ISPASS57527.2023.00036.
- [3] J. Balma and A. W. Rodriguez, “How to Cool Chips with Lasers: Startup Plans to Convert Heat to Light in Data-Center Processors,” *IEEE Spectrum*, vol. 62, no. 11, pp. 40–45, 2025, doi: 10.1109/MSPEC.2025.11237236.
- [4] J. Kong, S. W. Chung, and K. Skadron, “Recent thermal management techniques for microprocessors,” *ACM Computing Surveys*, vol. 44, no. 3, Art. no. 13, pp. 1–42, Jun. 2012, doi: 10.1145/2187671.2187675.
- [5] A. K. Coskun, T. S. Rosing, and K. Whisnant, “Temperature Aware Task Scheduling in MPSoCs,” in *Proc. 2007 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2007, pp. 1–6, doi: 10.1109/DATE.2007.364540.
- [6] A. K. Coskun, T. S. Rosing, K. A. Whisnant, and K. C. Gross, “Temperature-aware MPSoC scheduling for reducing hot spots and gradients,” in *Proc. 2008 Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2008, pp. 49–54, doi: 10.1109/ASPDAC.2008.4484002.
- [7] C.-Y. Lee, S.-J. Yang, and R.-G. Chang, “Thermal-Aware Scheduling Collaborating with OS and Architecture,” in *Proc. 2013 42nd International Conference on Parallel Processing (ICPP)*, 2013, pp. 1044–1051, doi: 10.1109/ICPP.2013.124.
- [8] A. Hankin, D. Werner, M. Amiraski, J. Sebot, K. Vaidyanathan, and M. Hempstead, “HotGauge: A Methodology for Characterizing Advanced Hotspots in Modern and Next Generation Processors,” in *Proc. 2021 IEEE International Symposium on Workload Characterization (IISWC)*, 2021, pp. 163–175, doi: 10.1109/IISWC53511.2021.00025.
- [9] A. Sridhar, A. Vincenzi, D. Atienza, and T. Brunschweiler, “3D-ICE: A compact thermal model for early-stage design of liquid-cooled ICs,” *IEEE Transactions on Computers*, vol. 63, no. 10, pp. 2576–2589, 2014.

- [10] T. E. Carlson, W. Heirman, and L. Eeckhout, “Sniper: Exploring the level of abstraction for scalable and accurate parallel multi-core simulations,” in *Proc. International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, Nov. 2011, pp. 52:1–52:12.
- [11] S. Xi, H. Jacobson, P. Bose, G.-Y. Wei, and D. Brooks, “Quantifying sources of error in McPAT and potential impacts on architectural studies,” in *Proc. International Symposium on High Performance Computer Architecture (HPCA)*, 2015. [Online]. Available: http://www.samxi.org/papers/xi_hpca2015.pdf
- [12] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/8a20a8621978632d76c43dfd28b67767-Paper.pdf